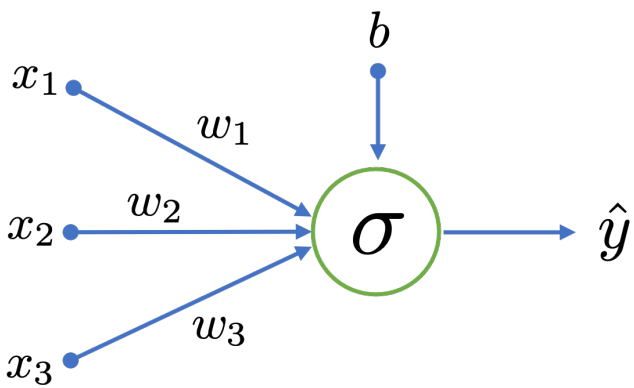
UNIVERSITY OF LEEDS

Multilayer networks

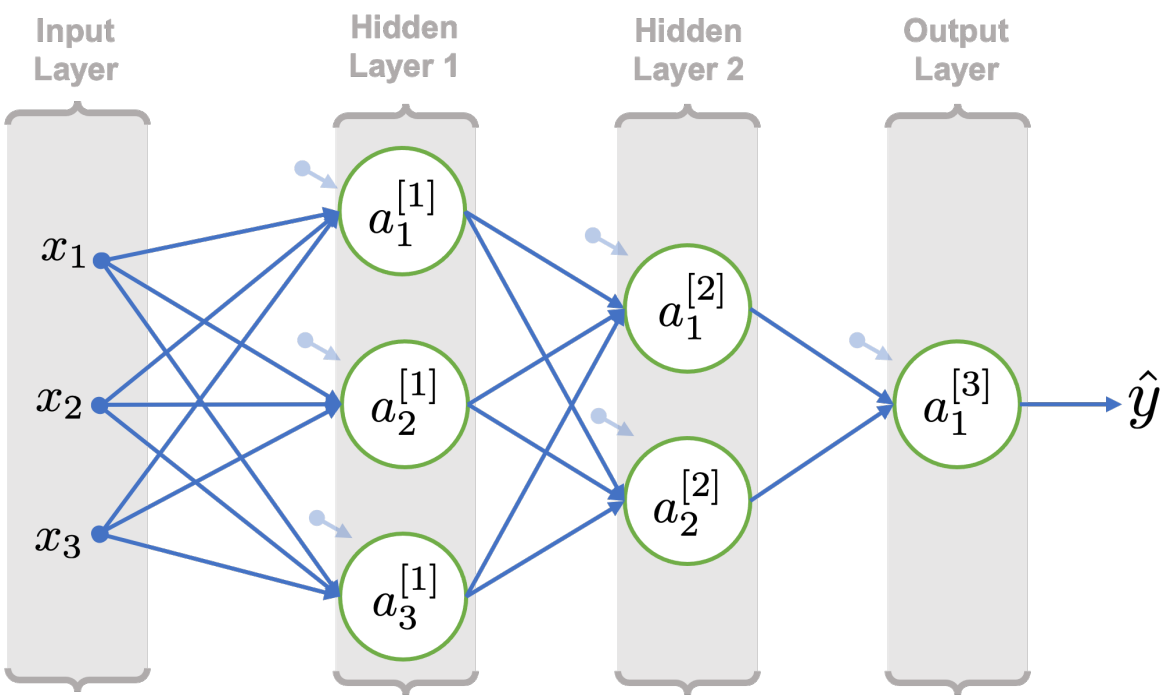
Deep Learning - COMP5203M

2022-23

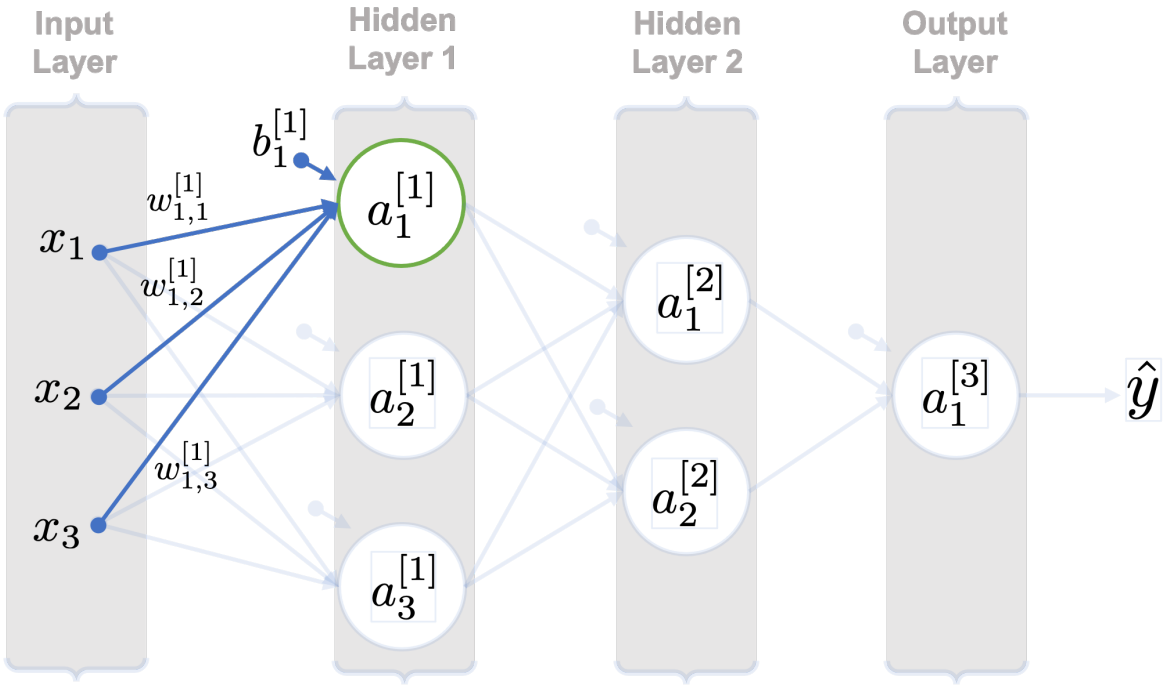
Perceptron



Multilayer Perceptron

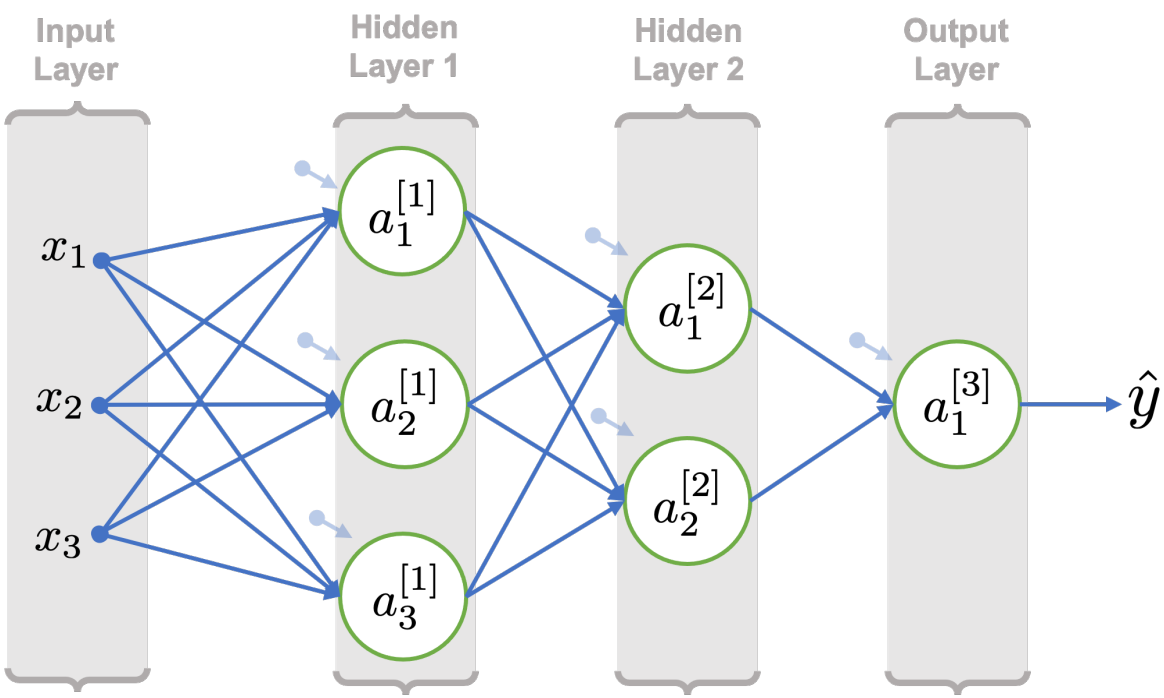


A single neuron activation



$$a_1^{[1]} = g({\boldsymbol{w}_1^{[1]}}^T \boldsymbol{x} + b_1^{[1]})$$

A single neuron activation (general)

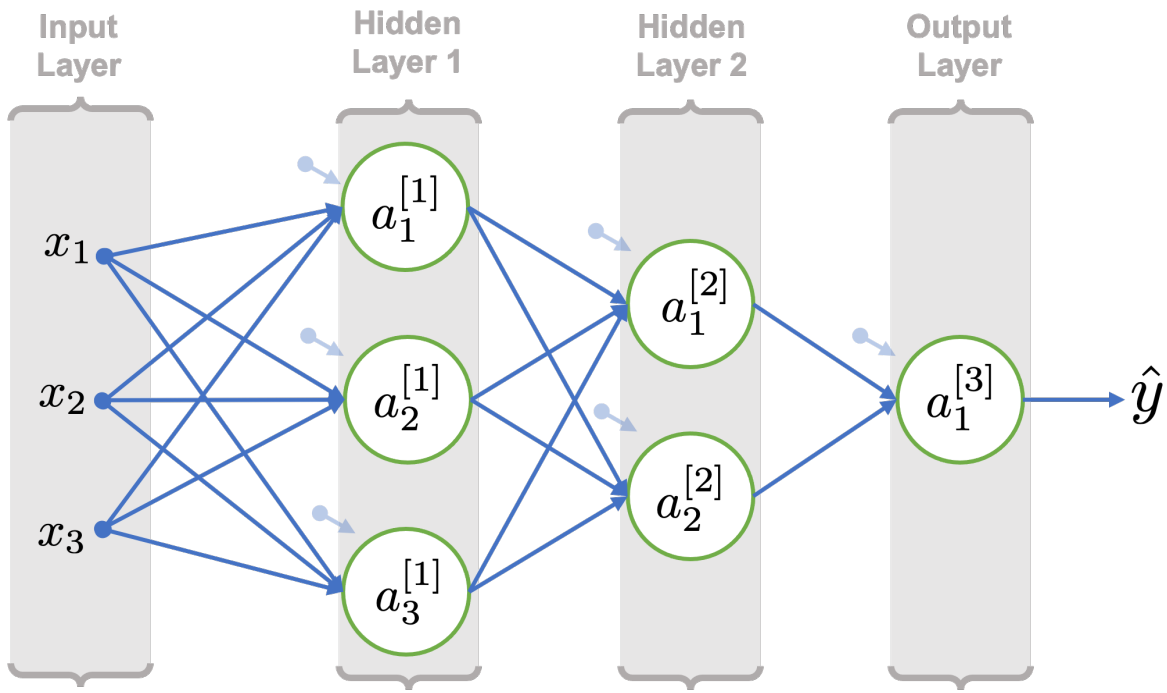


$$a_j^{[i]} = g(\boldsymbol{w}_j^{[i]^T} \boldsymbol{a}^{[i-1]} + b_j^{[i]})$$

Layer activations

$$\mathbf{W}^{[i]} = \begin{pmatrix} - & \boldsymbol{w}_1^{[i]T} & - \\ & \vdots & \\ - & \boldsymbol{w}_{n_i}^{[i]T} & - \end{pmatrix}$$

$$\boldsymbol{b}^{[i]} = \begin{pmatrix} b_1^{[i]} \\ \vdots \\ b_{n_i}^{[i]} \end{pmatrix}$$



$$\boldsymbol{a}^{[i]} = g(\mathbf{W}^{[i]}\boldsymbol{a}^{[i-1]} + \boldsymbol{b}^{[i]})$$

Activation for all training examples

$$\mathbf{X} = \mathbf{A}^{[0]} = \begin{pmatrix} | & & | \\ \boldsymbol{x}^{(1)}, & \ldots, & \boldsymbol{x}^{(m)} \\ | & & | \end{pmatrix}$$

$$\mathbf{A}^{[i]} = g(\mathbf{W}^{[i]}\mathbf{A}^{[i-1]} + \boldsymbol{b}^{[i]})$$

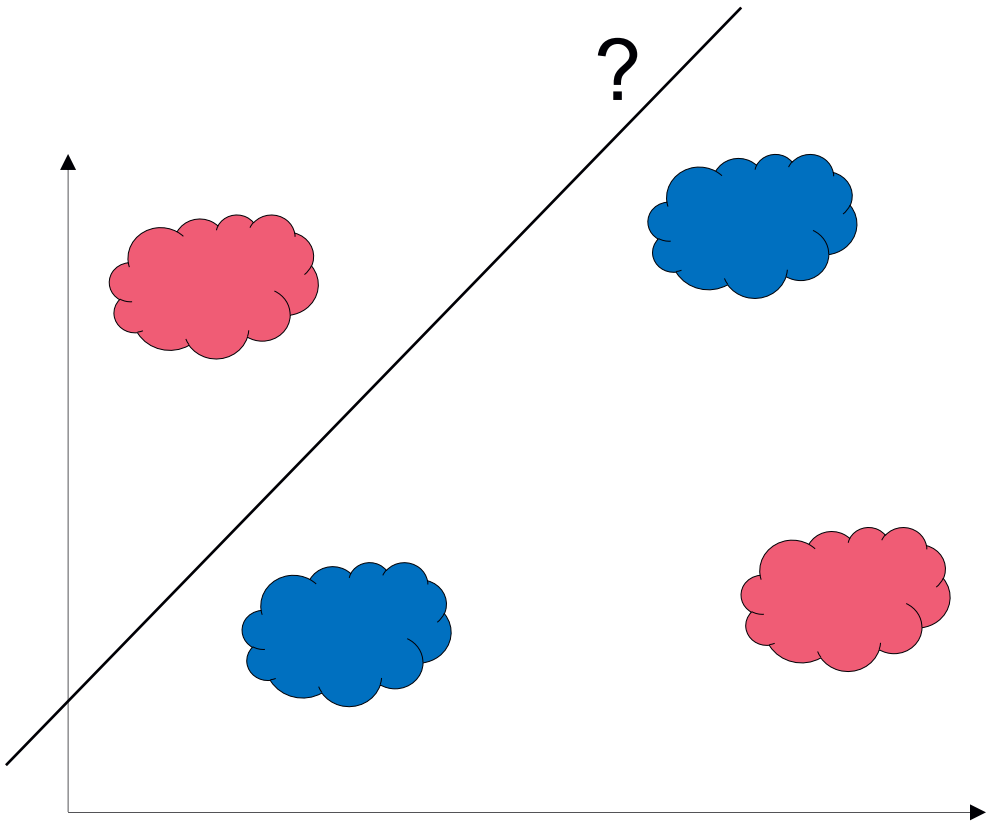
Activation for all training examples

$$\mathbf{A}^{[i]} = g(\mathbf{W}^{[i]}\mathbf{A}^{[i-1]} + \boldsymbol{b}^{[i]})$$

$$\left.\begin{pmatrix} - & \boldsymbol{w}_1^{[i]T} & - \\ & \vdots & \\ - & \boldsymbol{w}_{n_i}^{[i]T} & - \end{pmatrix} \begin{pmatrix} | & & | \\ \boldsymbol{a}^{[i-1](1)}, & \ldots, & \boldsymbol{a}^{[i-1](m)} \\ | & & | \end{pmatrix} + \begin{pmatrix} b_1^{[i]} \\ \vdots \\ b_{n_i}^{[i]} \end{pmatrix}\right\} \xrightarrow{g(\cdot)}$$

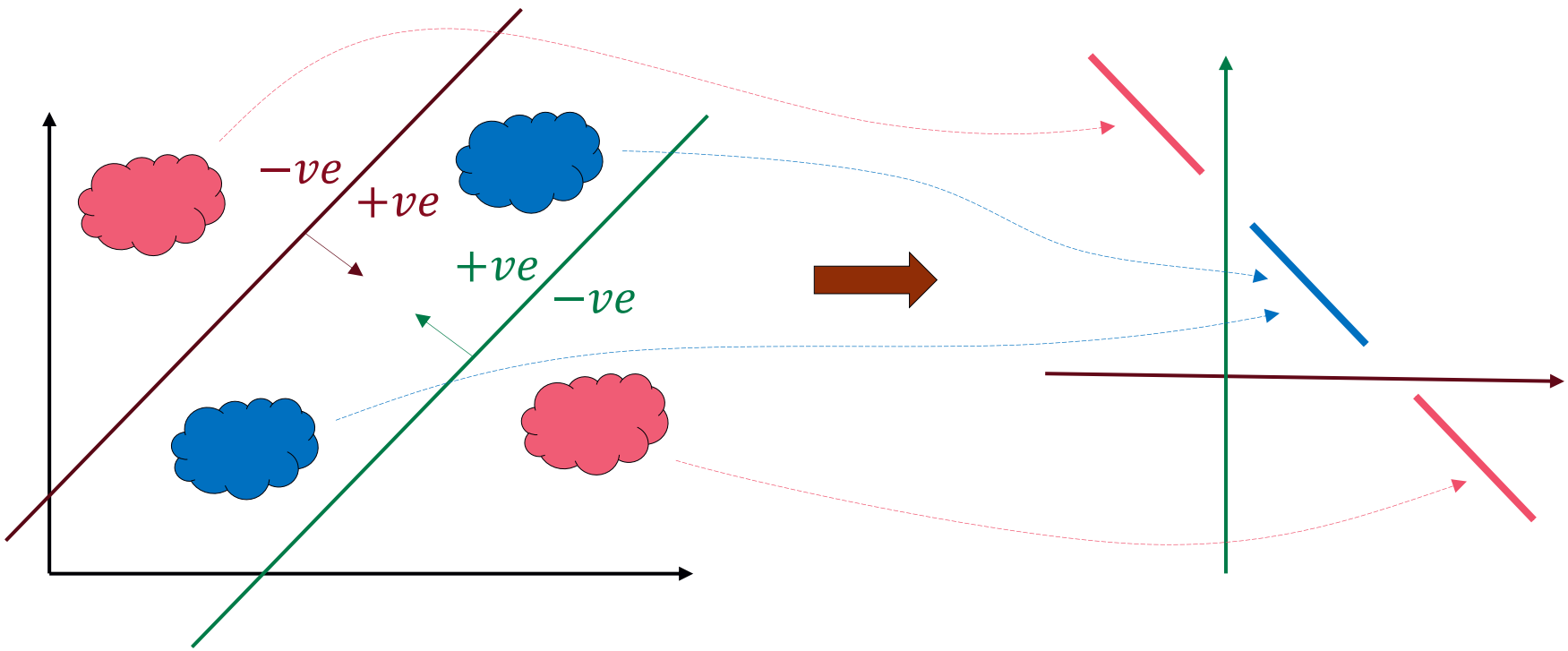
$$\mathbf{A}^{[i]} = \begin{pmatrix} | & & | \\ \boldsymbol{a}^{[i](1)}, & \ldots, & \boldsymbol{a}^{[i](m)} \\ | & & | \end{pmatrix}$$

Consider this binary classification problem



Solution – add another layer?

Still not separable!



Why doesn't this work?

$$\boldsymbol{z}_1 = \boldsymbol{W}_1\boldsymbol{x} + \boldsymbol{b}_1$$ Layer 1

$$\boldsymbol{z}_2 = \boldsymbol{W}_2\boldsymbol{z}_1 + \boldsymbol{b}_2$$ Layer 2

But this reduces to:

$$\boldsymbol{z}_2 = \boldsymbol{W}_2(\boldsymbol{W}_1\boldsymbol{x} + \boldsymbol{b}_1) + \boldsymbol{b}_2 = \boldsymbol{W}_1\boldsymbol{W}_2 x + \boldsymbol{W}_2\boldsymbol{b}_1 + \boldsymbol{b}_2$$

Which is again an affine function (i.e. a single layer)

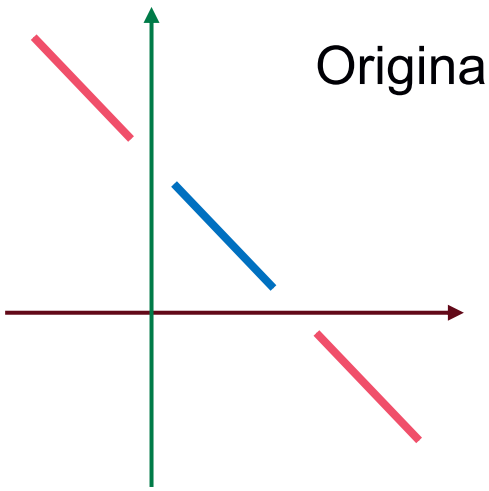
$$\boldsymbol{z}_2 = \underbrace{\boldsymbol{W}_1\boldsymbol{W}_2}_{\boldsymbol{W}_3}\boldsymbol{x} + \underbrace{\boldsymbol{W}_2\boldsymbol{b}_1 + \boldsymbol{b}_2}_{\boldsymbol{b}_3}$$

An affine function of an affine function is affine

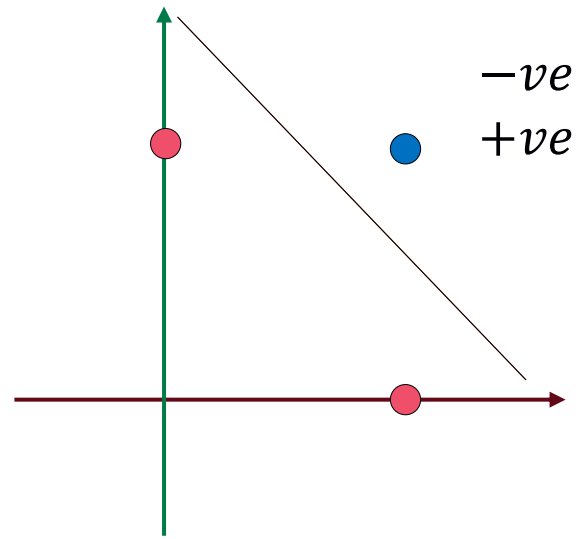
Solution

Add a non-linear function after the 1st layer

Known as an *activation* function

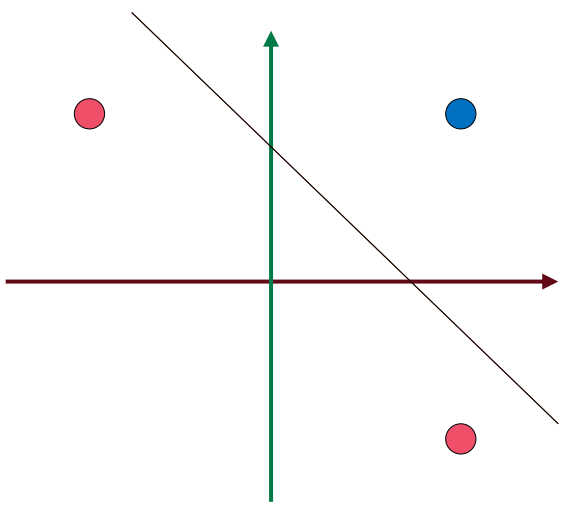


Original



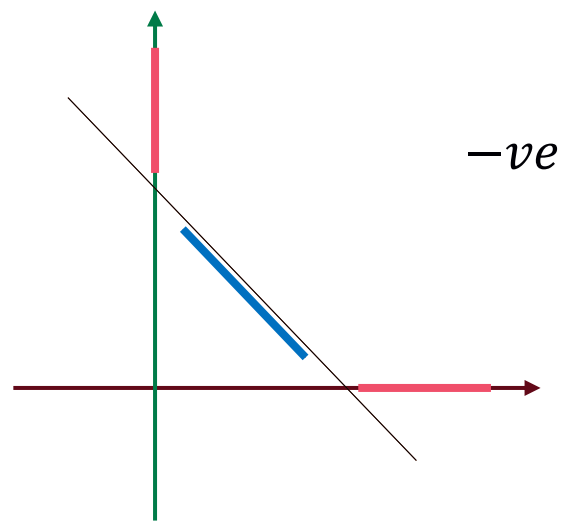
$-ve \rightarrow 0$

$+ve \rightarrow 1$



$-ve \rightarrow -1$

$+ve \rightarrow 1$



$-ve \rightarrow 0$

Logistic activation function

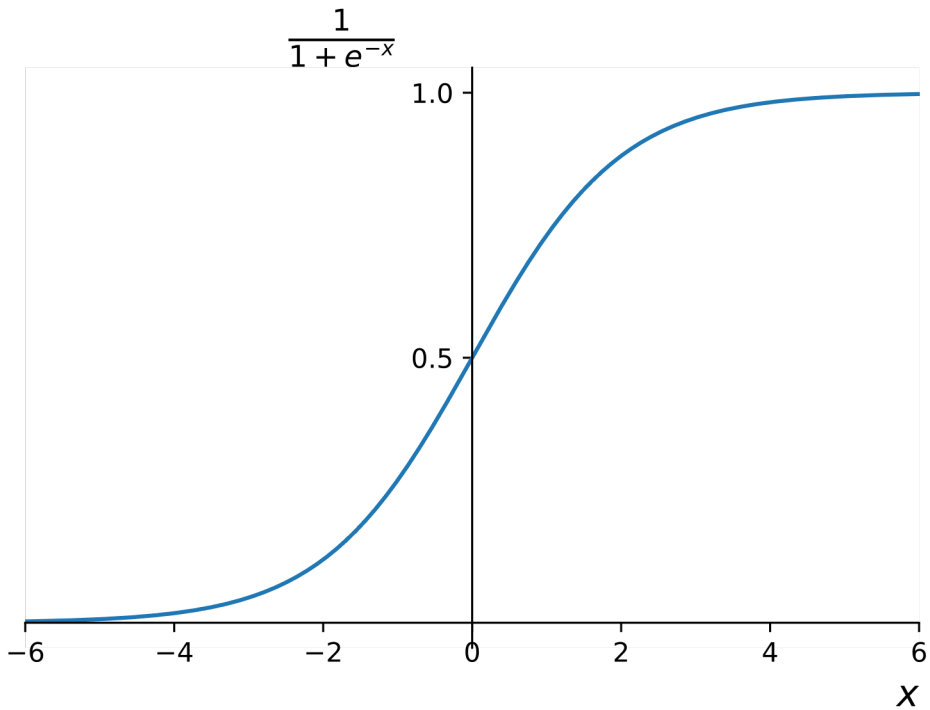
$$f(x) = \frac{1}{1 + e^{-x}}$$

$$f(x) \in [0,1]$$

Smooth approximation to:

$-ve \rightarrow 0$

$+ve \rightarrow 1$



Often referred to as a *sigmoid* function because of the "S" shape (from the Greek 'word-final' variant of sigma: ς).

Hyperbolic tangent (tanh) activation function

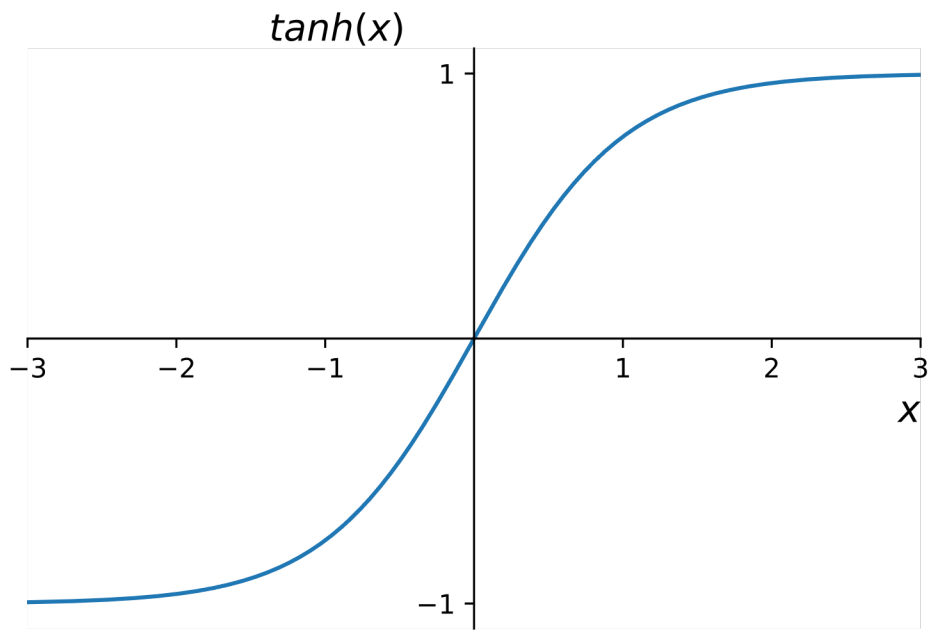
$$\tanh(x) = \frac{(e^{2x} - 1)}{(e^{2x} + 1)}$$

$$\tanh(x) \in [-1, 1]$$

Smooth approximation to:

$-ve \rightarrow -1$

$+ve \rightarrow 1$

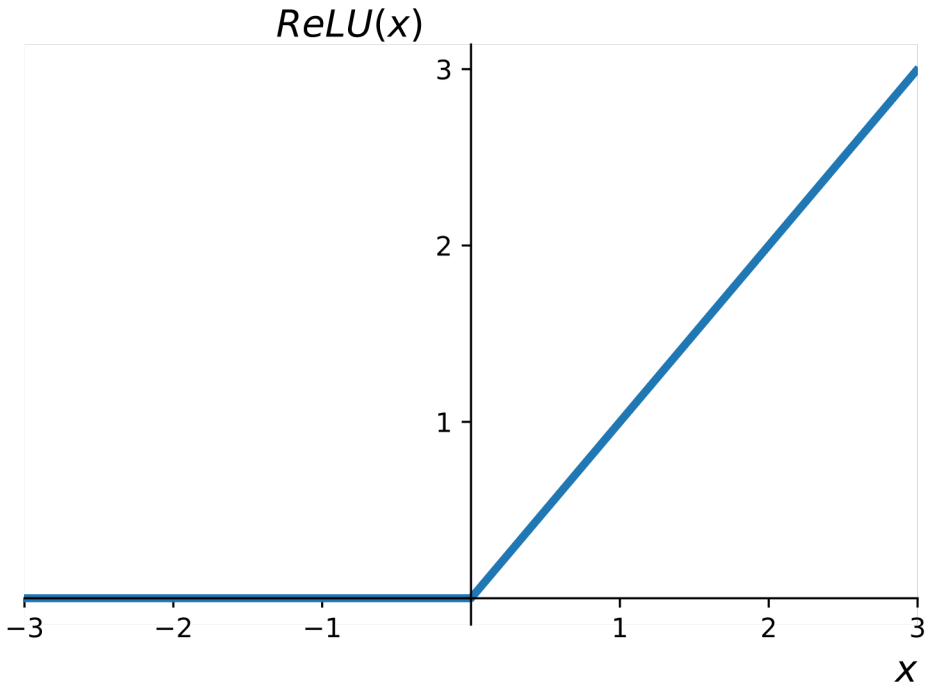


Rectified Linear Unit (ReLU) activation function

$$\text{ReLU}(x) = \max(0, x)$$

$$\text{ReLU}(x) \in [0, \infty)$$

$-ve \rightarrow 0$



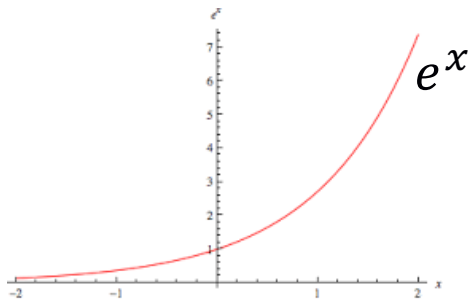
Smooth approximations include *GeLU* and *swish*

Defining a probability distribution over class labels

Softmax maps confidence values $\mathbf{z} = (z_1 \cdots z_K)$ to the interval $[0,1]$ such that they sum to 1 and keep numerical order.

$$\sigma(\boldsymbol{z})_c = \frac{e^{z_c}}{\sum_{k=1}^{K} e^{z_k}}$$

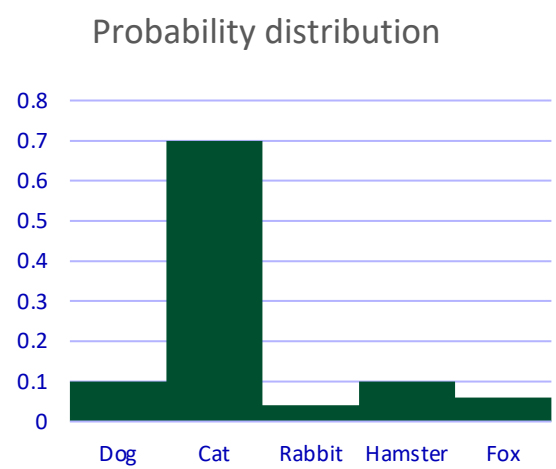
Normalisation ensures outputs sum to one



All input values become +ve prior to normalisation

$$\begin{bmatrix} 2.1 \\ 1.2 \\ -1.5 \end{bmatrix} \rightarrow \begin{bmatrix} 0.70 \\ 0.28 \\ 0.02 \end{bmatrix}$$

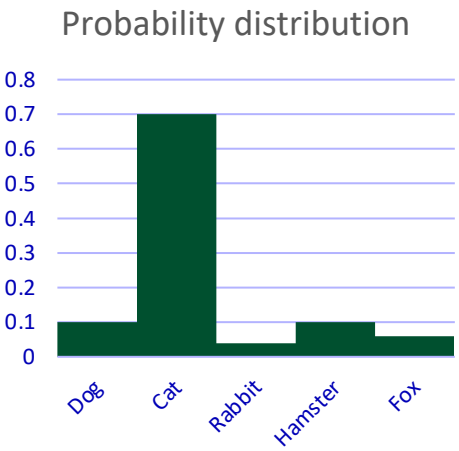
sums to one



A loss function for classification tasks

Focus on classification tasks where the output is a probability distribution

Classifier



Parameter values $\boldsymbol{\theta}$

model parameter values
(weights and biases)

The Loss Function (all inputs)

$$L(\boldsymbol{\theta}) = \prod_{i=1}^{m} p\left(y^{(i)} | \boldsymbol{x}^{(i)}; \boldsymbol{\theta}\right)$$

$$\boldsymbol{\theta}_{ML} = \arg\max_{\boldsymbol{\theta}} \prod_{i=1}^{m} p(y^{(i)} | \boldsymbol{x}^{(i)}; \boldsymbol{\theta})$$

The Loss Function (all inputs)

$$L(\boldsymbol{\theta}) = \prod_{i=1}^{m} p\left(y^{(i)} | \boldsymbol{x}^{(i)}; \boldsymbol{\theta}\right)$$

Hard to optimize

$$\boldsymbol{\theta}_{ML} = \arg\max_{\boldsymbol{\theta}} \prod_{i=1}^{m} p(y^{(i)} | \boldsymbol{x}^{(i)}; \boldsymbol{\theta})$$

The Loss Function (all inputs)

$$L(\boldsymbol{\theta}) = \prod_{i=1}^{m} p\left(y^{(i)} | \boldsymbol{x}^{(i)}; \boldsymbol{\theta}\right)$$

Hard to optimize

$\log(\cdot)$

$$\boldsymbol{\theta}_{ML} = \arg\max_{\boldsymbol{\theta}} \prod_{i=1}^{m} p(y^{(i)} | \boldsymbol{x}^{(i)}; \boldsymbol{\theta})$$

$$l(\boldsymbol{\theta}) = \log L(\boldsymbol{\theta}) = \log\left(\prod_{i=1}^{m} p(y^{(i)} | \boldsymbol{x}^{(i)}; \boldsymbol{\theta})\right) = \sum_{i=1}^{m} \log(p(y^{(i)} | \boldsymbol{x}^{(i)}; \boldsymbol{\theta})$$

The Loss Function (all inputs)

$$L(\boldsymbol{\theta}) = \prod_{i=1}^{m} p\left(y^{(i)} | \boldsymbol{x}^{(i)}; \boldsymbol{\theta}\right)$$

Hard to optimize

$\log(\cdot)$

$$\boldsymbol{\theta}_{ML} = \arg\max_{\boldsymbol{\theta}} \prod_{i=1}^{m} p(y^{(i)} | \boldsymbol{x}^{(i)}; \boldsymbol{\theta})$$

$$l(\boldsymbol{\theta}) = \log L(\boldsymbol{\theta}) = \log\left(\prod_{i=1}^{m} p(y^{(i)} | \boldsymbol{x}^{(i)}; \boldsymbol{\theta})\right) = \sum_{i=1}^{m} \log(p(y^{(i)} | \boldsymbol{x}^{(i)}; \boldsymbol{\theta})$$

Easier to optimize

Gradient Descent

Cost function

$$J(\boldsymbol{\theta}) = \sum_{i=1}^{m} -\log(p(y^{(i)}|\boldsymbol{x}^{(i)};\boldsymbol{\theta}))$$

$$\boldsymbol{\theta} := \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$$

Cost function gradient

$$\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) = \begin{pmatrix} \frac{\partial J}{\partial \boldsymbol{w}} \\ \frac{\partial J}{\partial \boldsymbol{b}} \end{pmatrix}$$

Stochastic Gradient Descent (SGD)

Estimate the gradient on a *minibatch* (small subset) of the data in each iteration of gradient descent.

The size of the minibatch $m' \ll m$ (typically 1-100)

After all examples have been used once, this marks the end of an *epoch*

SGD often generalises well to test cases because of the randomness introduced by small minibatches.

Larger minibatches reduce compute-time.

Terminology

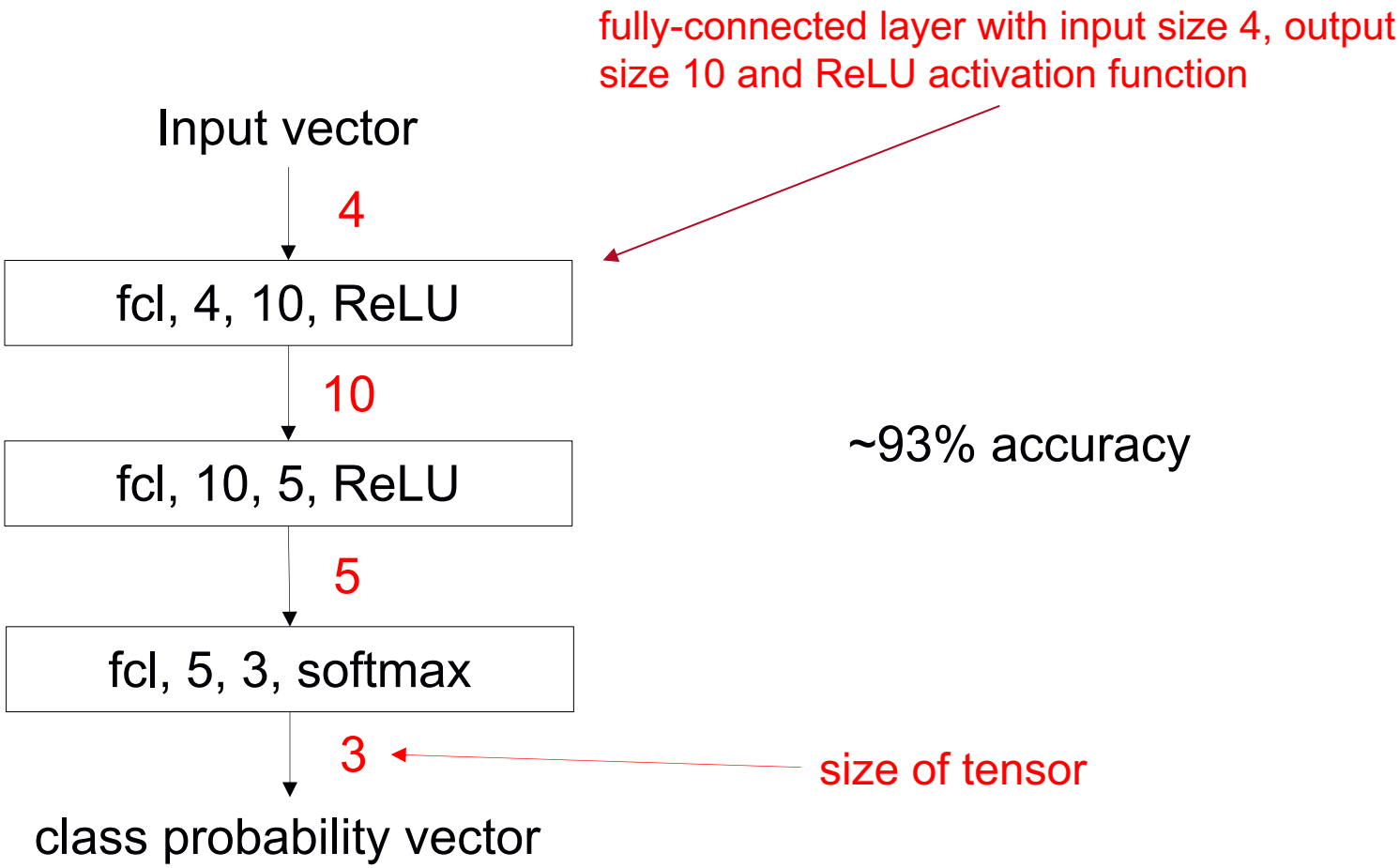
Multilayer Perceptron (MLP): a network with multiple 'linear' layers (including activation functions)

The layers are referred to as *Linear* layers in PyTorch and Tensorflow, and *Dense* layers in Keras.

Also referred to as *fully-connected layers (fcl* in network diagrams*)*

Every output value is a function of every input value

3-layer architecture for iris dataset



Training a classifier

e.g., 4-tuples with Iris dataset, images

Training set D of inputs $\{\boldsymbol{x}^{(1)}, \cdots, \boldsymbol{x}^{(m)}\}$, with class labels $\{y^{(1)}, \cdots, y^{(m)}\}$

Multilayer network with N layers and parameters $\theta = \{(\mathbf{W}_i, \boldsymbol{b}_i)\}_{1 \le i \le N}$

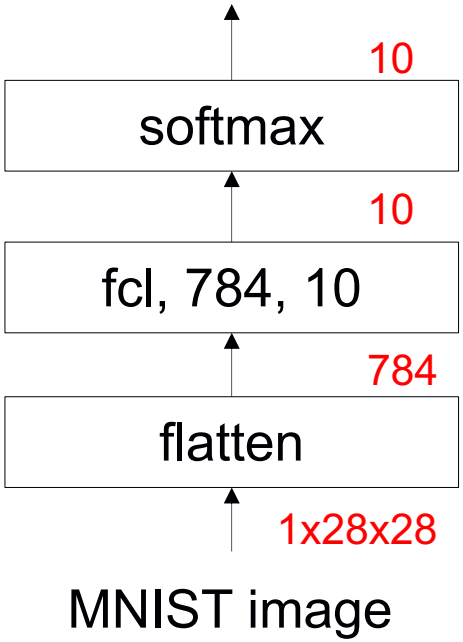
Define loss function $L(\theta, D)$ ← captures performance of the network

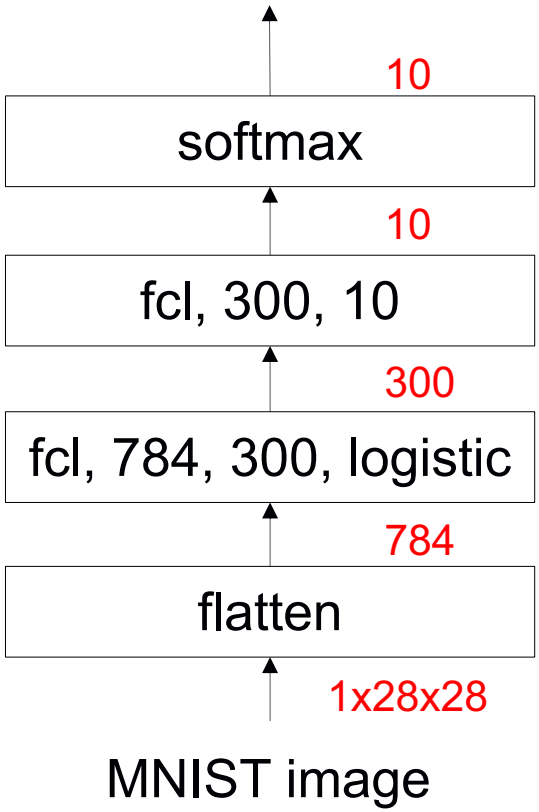
Minimise L over the values of θ, assuming D fixed

Evaluate performance on test set (e.g., accuracy)

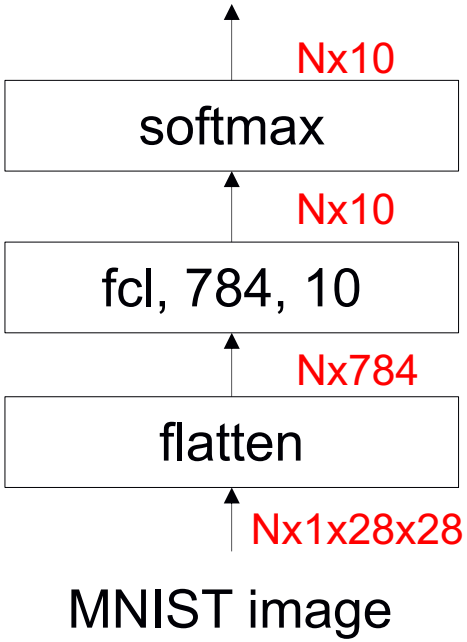
Normally split the training set into two: one subset to learn the parameters and the other (the *validation set*) to test generalization error. Thus, can try out different network architectures (e.g., number of layers, sizes of layers, choice of activation functions) without touching the test set.

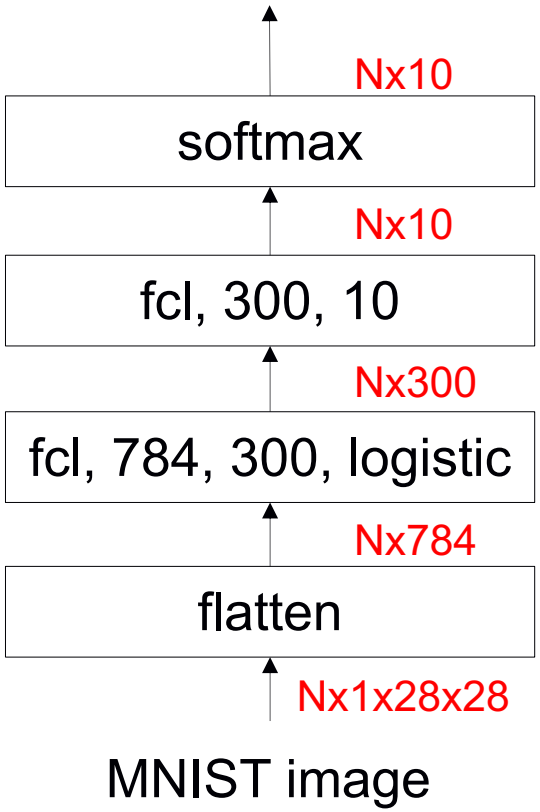
Example on MNIST





Processing minibatches

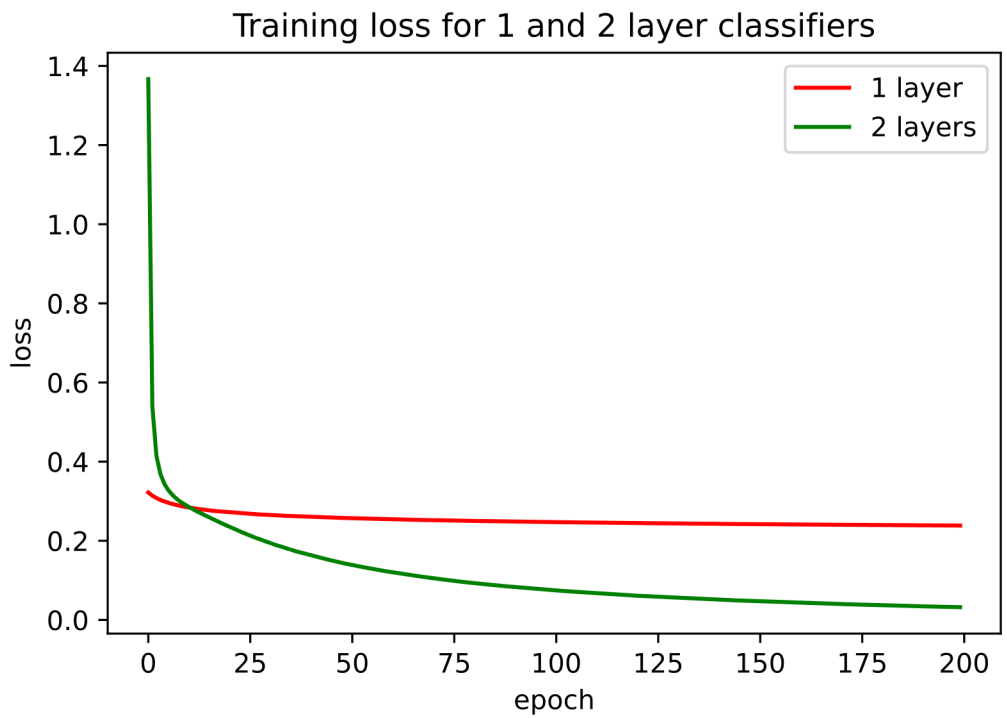




To compute loss, apply network to a minibatch of N examples.

Do this efficiently by taking through together, as shown.

Performance on MNIST



Sigmoid activation function

200 epochs

1 fcl layer accuracy: 92.6%

2 fcl layers accuracy: 97.8%

Momentum in gradient descent

Gradient descent simply updates $\boldsymbol{\theta}$ according to the gradient of the output at its current value:

$$\boldsymbol{\theta}' = \boldsymbol{\theta} - \epsilon \nabla_{\boldsymbol{\theta}} f(\boldsymbol{\theta})$$

The idea of adding *momentum* is to think of $\boldsymbol{\theta}$ as having a 'velocity' . On each iteration it slows down a bit and gets nudged in the direction of the negative gradient (the direction of fastest decrease in $f(\boldsymbol{\theta})$)

$$\mathbf{v}' = \alpha \mathbf{v} - \epsilon \nabla_{\boldsymbol{\theta}} f(\boldsymbol{\theta}) \qquad \alpha \in [0,1)$$

interval excludes 1

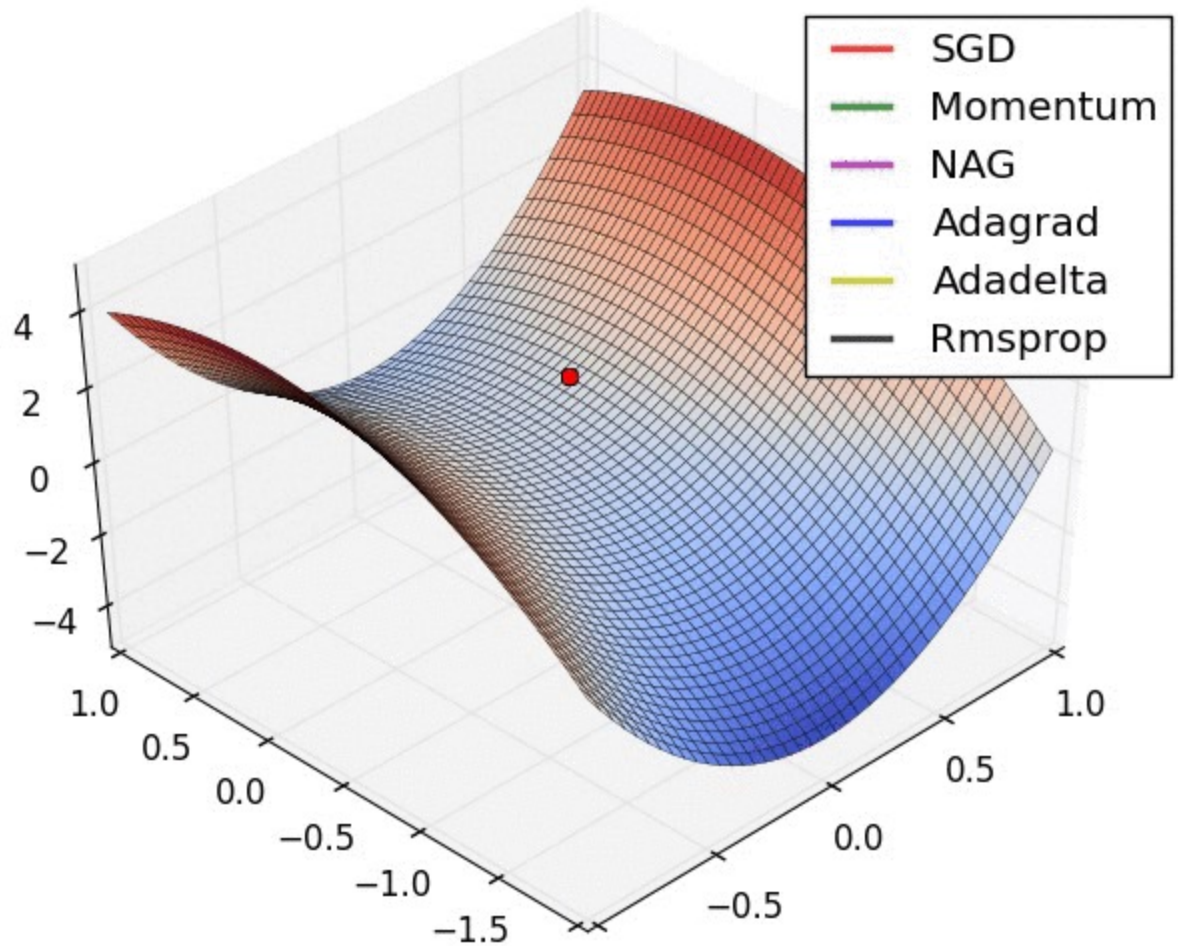
The update to $\boldsymbol{\theta}$ is now:

$$\boldsymbol{\theta}' = \boldsymbol{\theta} + \mathbf{v}'$$

Nesterov momentum is a slight variation on this. The gradient is evaluated after the current velocity is applied:

$$\mathbf{v}' = \alpha \mathbf{v} - \epsilon \nabla_{\boldsymbol{\theta}} f(\boldsymbol{\theta} + \alpha \mathbf{v}) \qquad \alpha \in [0,1)$$

Aside: Visualisation of different optimisers



https://aihub.cloud.google.com/p/products%2Fa1577efd-2ed8-4b9d-964e-e87e2076dc59

UNIVERSITY OF LEEDS

Backpropagation

Deep Learning - COMP5203M

2022-23

MLPs as Composite Functions

$$\boldsymbol{y} = (f_K \circ f_{K-1} \circ \cdots \circ f_1)(\boldsymbol{x}) = f_K(f_{k-1}(\ldots(f_1(\boldsymbol{x}))\ldots))$$

MLPs as Composite Functions

$$\boldsymbol{y} = (f_K \circ f_{K-1} \circ \cdots \circ f_1)(\boldsymbol{x}) = f_K(f_{k-1}(\ldots(f_1(\boldsymbol{x}))\ldots))$$

$$\boldsymbol{f}_0 := \boldsymbol{x}$$

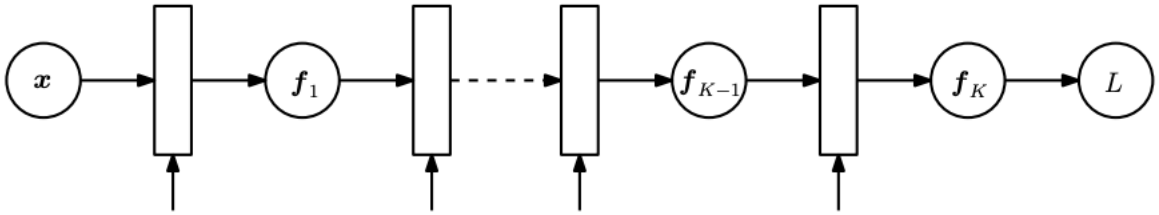
$$\boldsymbol{f}_i := g(\mathbf{W}^{[\mathbf{i}-\mathbf{1}]}\boldsymbol{f}_{i-1} + \boldsymbol{b}_1), i = 1, \ldots, K$$

MLPs as Composite Functions

$$\boldsymbol{y} = (f_K \circ f_{K-1} \circ \cdots \circ f_1)(\boldsymbol{x}) = f_K(f_{k-1}(\ldots(f_1(\boldsymbol{x}))\ldots))$$

$$\boldsymbol{f}_0 := \boldsymbol{x}$$

$$\boldsymbol{f}_i := g(\mathbf{W}^{[\mathbf{i}-\mathbf{1}]}\boldsymbol{f}_{i-1} + \boldsymbol{b}_1), i = 1, \ldots, K$$

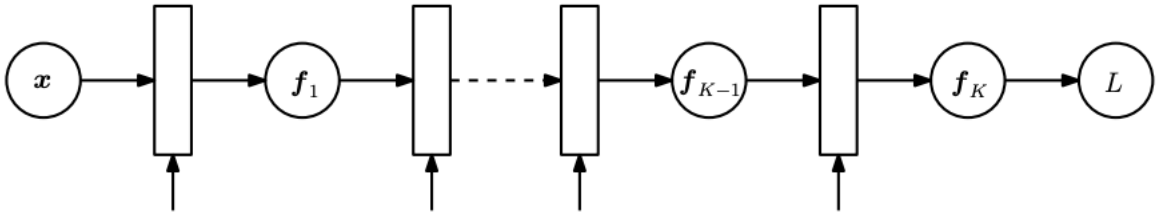


MLPs as Composite Functions

$$\boldsymbol{y} = (f_K \circ f_{K-1} \circ \cdots \circ f_1)(\boldsymbol{x}) = f_K(f_{k-1}(\ldots(f_1(\boldsymbol{x}))\ldots))$$

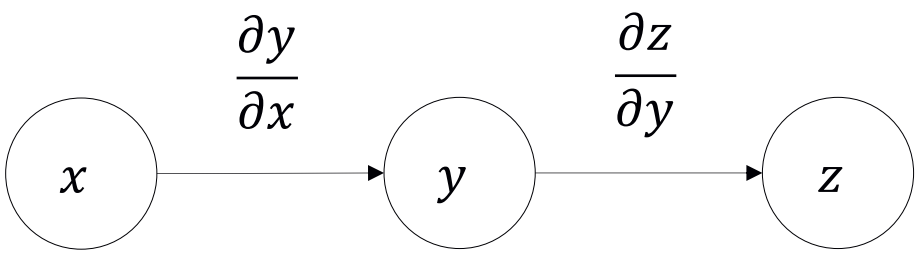
$$\boldsymbol{f}_0 := \boldsymbol{x}$$

$$\boldsymbol{f}_i := g(\mathbf{W}^{[\mathbf{i-1}]}\boldsymbol{f}_{i-1} + \boldsymbol{b}_1), i = 1, \ldots, K$$



$$L(\boldsymbol{\theta}) = \Gamma(\boldsymbol{y}, f_K(\boldsymbol{x}, \boldsymbol{\theta}))$$

Chain rule



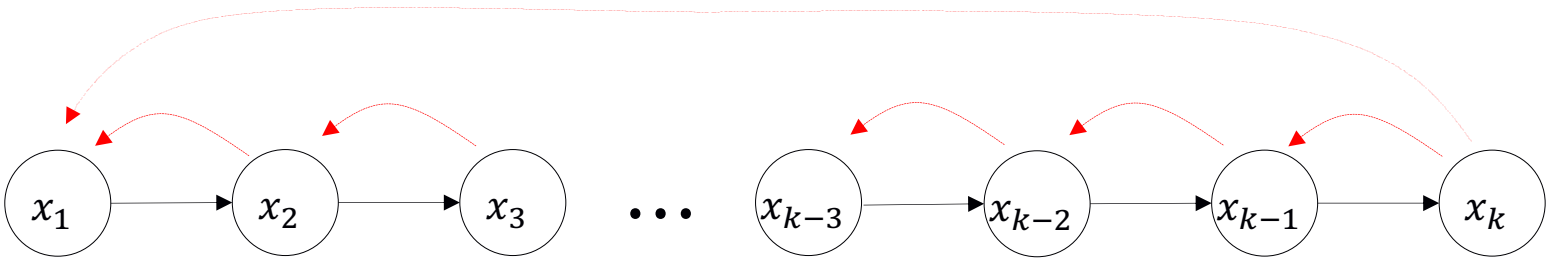
$$\frac{\partial z}{\partial x} = \frac{\partial z}{\partial y}\frac{\partial y}{\partial x}$$

Chain rule

if a car travels twice as fast as a bicycle and the bicycle is four times as fast as a walking man, then the car travels 2 × 4 = 8 times as fast as the man.

George F. Simmons, Calculus with Analytic Geometry, McGraw-Hill, New York (1985), p93

For simple 'linear' computation graphs



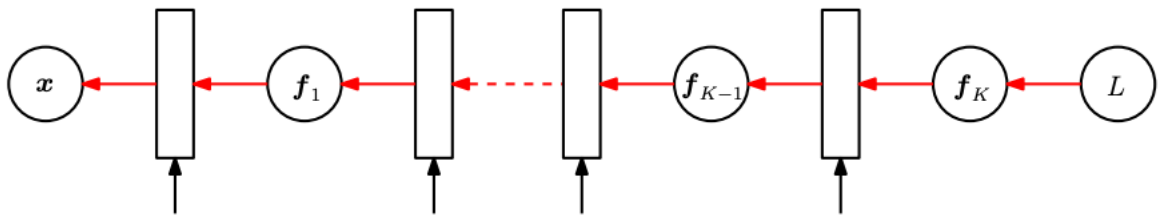
By repeated use of the chain rule, we get:

$$\frac{\partial x_k}{\partial x_1} = \frac{\partial x_2}{\partial x_1} \cdots \frac{\partial x_{k-1}}{\partial x_{k-2}} \frac{\partial x_k}{\partial x_{k-1}}$$

MLPs as Composite Functions

$$\boldsymbol{y} = (f_K \circ f_{K-1} \circ \cdots \circ f_1)(\boldsymbol{x}) = f_K(f_{k-1}(\ldots(f_1(\boldsymbol{x}))\ldots))$$

$$L(\boldsymbol{\theta}) = \Gamma(\boldsymbol{y}, f_K(\boldsymbol{x}, \boldsymbol{\theta}))$$



$$\frac{\partial L}{\partial \boldsymbol{\theta}_{K-1}} = \frac{\partial L}{\partial \boldsymbol{f}_K} \frac{\partial \boldsymbol{f}_K}{\partial \boldsymbol{\theta}_{K-1}}$$

$$\frac{\partial L}{\partial \boldsymbol{\theta}_{K-2}} = \frac{\partial L}{\partial \boldsymbol{f}_K} \boxed{\frac{\partial \boldsymbol{f}_K}{\partial \boldsymbol{f}_{K-1}} \frac{\partial \boldsymbol{f}_{K-1}}{\partial \boldsymbol{\theta}_{K-2}}}$$

$$\frac{\partial L}{\partial \boldsymbol{\theta}_{K-3}} = \frac{\partial L}{\partial \boldsymbol{f}_K} \frac{\partial \boldsymbol{f}_K}{\partial \boldsymbol{f}_{K-1}} \boxed{\frac{\partial \boldsymbol{f}_{K-1}}{\partial \boldsymbol{f}_{K-2}} \frac{\partial \boldsymbol{f}_{K-2}}{\partial \boldsymbol{\theta}_{K-3}}}$$

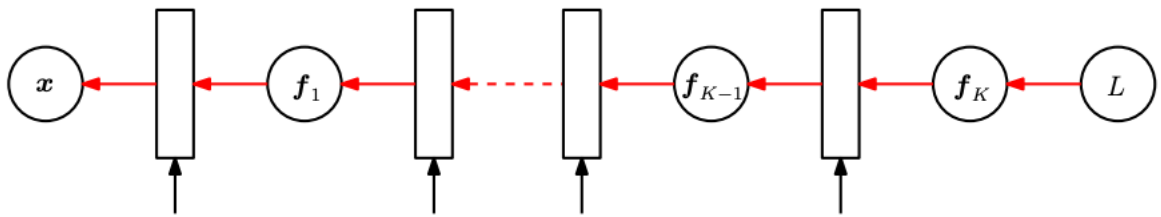
$$\frac{\partial L}{\partial \boldsymbol{\theta}_i} = \frac{\partial L}{\partial \boldsymbol{f}_K} \frac{\partial \boldsymbol{f}_K}{\partial \boldsymbol{f}_{K-1}} \cdots \boxed{\frac{\partial \boldsymbol{f}_{i+2}}{\partial \boldsymbol{f}_{i+1}} \frac{\partial \boldsymbol{f}_{i+1}}{\partial \boldsymbol{\theta}_i}}$$

Deisenroth, Marc Peter, A. Aldo Faisal, and Cheng Soon Ong. *Mathematics for machine learning*. Cambridge University Press, 2020.

MLPs as Composite Functions

$$\boldsymbol{y} = (f_K \circ f_{K-1} \circ \cdots \circ f_1)(\boldsymbol{x}) = f_K(f_{k-1}(\ldots(f_1(\boldsymbol{x}))\ldots))$$

$$L(\boldsymbol{\theta}) = \Gamma(\boldsymbol{y}, f_K(\boldsymbol{x}, \boldsymbol{\theta}))$$



$$\frac{\partial L}{\partial \boldsymbol{\theta}_{K-1}} = \frac{\partial L}{\partial \boldsymbol{f}_K} \frac{\partial \boldsymbol{f}_K}{\partial \boldsymbol{\theta}_{K-1}}$$

$$\frac{\partial L}{\partial \boldsymbol{\theta}_{K-2}} = \frac{\partial L}{\partial \boldsymbol{f}_K} \boxed{\frac{\partial \boldsymbol{f}_K}{\partial \boldsymbol{f}_{K-1}} \frac{\partial \boldsymbol{f}_{K-1}}{\partial \boldsymbol{\theta}_{K-2}}}$$

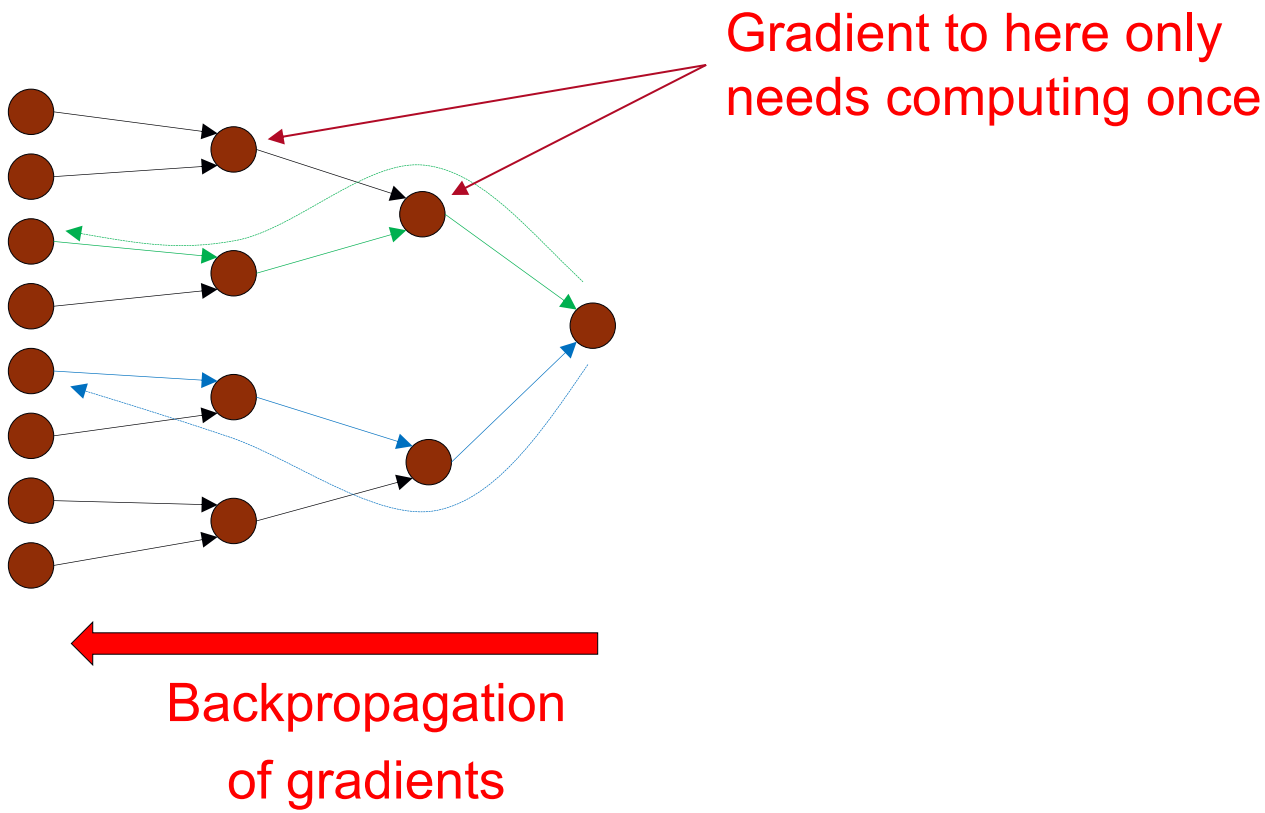
$$\frac{\partial L}{\partial \boldsymbol{\theta}_{K-3}} = \frac{\partial L}{\partial \boldsymbol{f}_K} \frac{\partial \boldsymbol{f}_K}{\partial \boldsymbol{f}_{K-1}} \boxed{\frac{\partial \boldsymbol{f}_{K-1}}{\partial \boldsymbol{f}_{K-2}} \frac{\partial \boldsymbol{f}_{K-2}}{\partial \boldsymbol{\theta}_{K-3}}}$$

$$\frac{\partial L}{\partial \boldsymbol{\theta}_i} = \frac{\partial L}{\partial \boldsymbol{f}_K} \frac{\partial \boldsymbol{f}_K}{\partial \boldsymbol{f}_{K-1}} \cdots \boxed{\frac{\partial \boldsymbol{f}_{i+2}}{\partial \boldsymbol{f}_{i+1}} \frac{\partial \boldsymbol{f}_{i+1}}{\partial \boldsymbol{\theta}_i}}$$

Backpropagation: application of chain rule of differentiation

Deisenroth, Marc Peter, A. Aldo Faisal, and Cheng Soon Ong. *Mathematics for machine learning*. Cambridge University Press, 2020.

For any tree-structured computation graph, we can backpropagate gradients in this way, using the chain rule along the unique paths from output (root) to inputs (leaves).

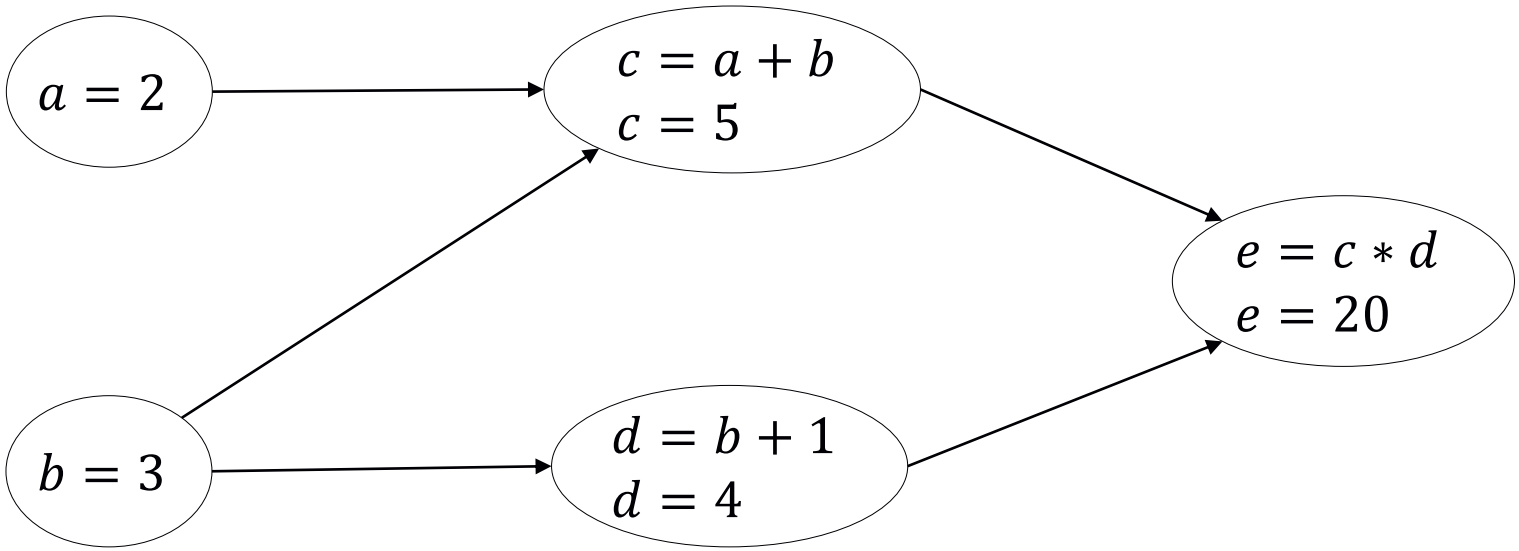


Computing partial derivatives efficiently

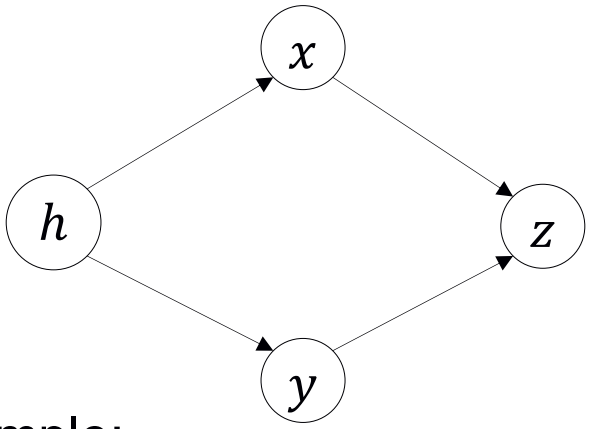
or *computational graph*

Represent function as a *computation graph*

$$e = (a + b) * (b + 1) \quad \text{for } a = 2,\ b = 3$$



Multi-variable functions



With multi-variable functions, there may be more than one 'chain' from some nodes to the output.

Generalisation of the chain rule for this example:

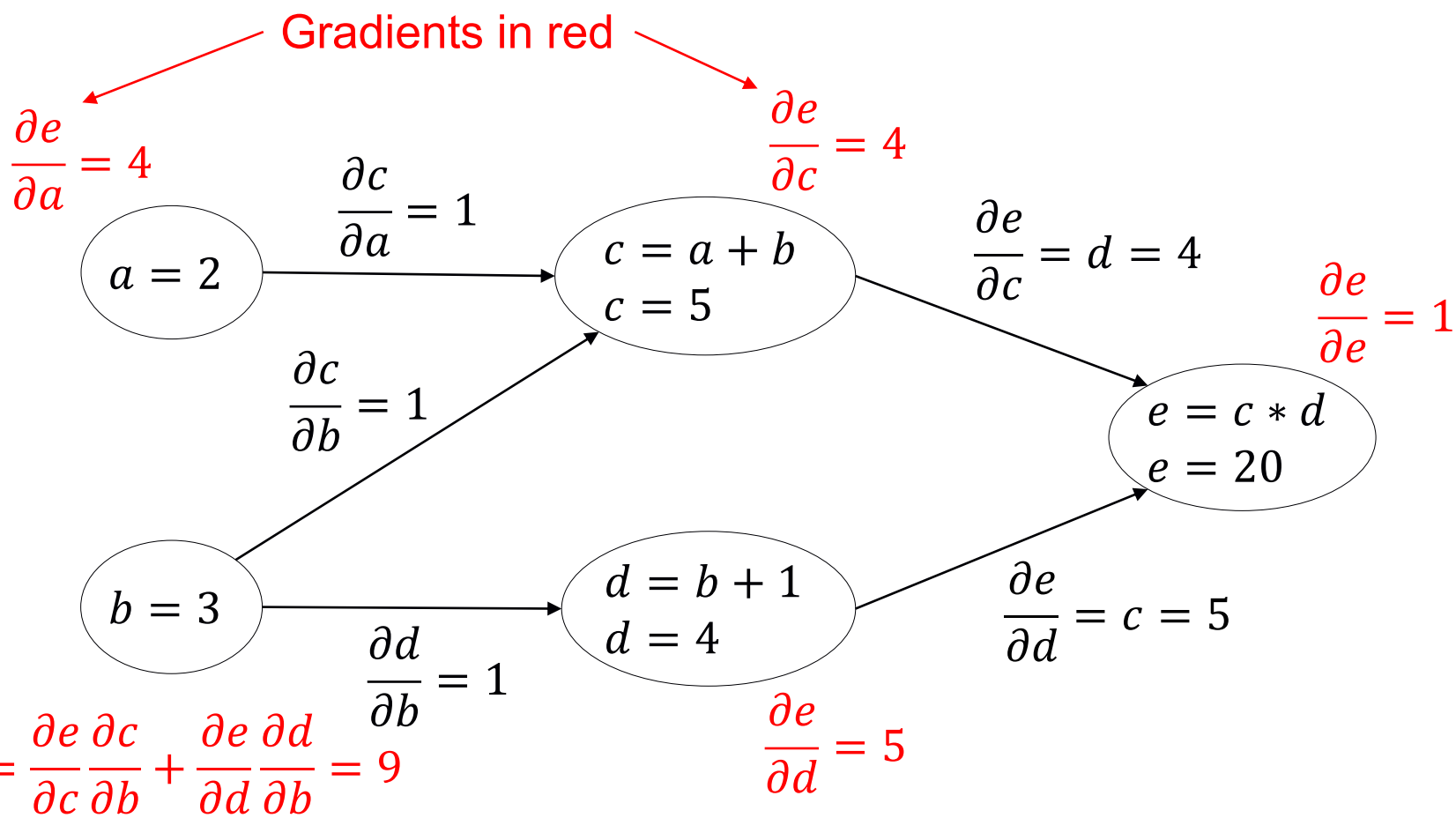
$$\frac{\partial z}{\partial h} = \frac{\partial z}{\partial x}\frac{\partial x}{\partial h} + \frac{\partial z}{\partial y}\frac{\partial y}{\partial h}$$

Intuitively, we are adding the products from the two chains.

In general, the gradient at each node is the sum of <u>all</u> chain-products leading to the output.

During backpropagation, we accumulate (sum) these contributions to the gradient.

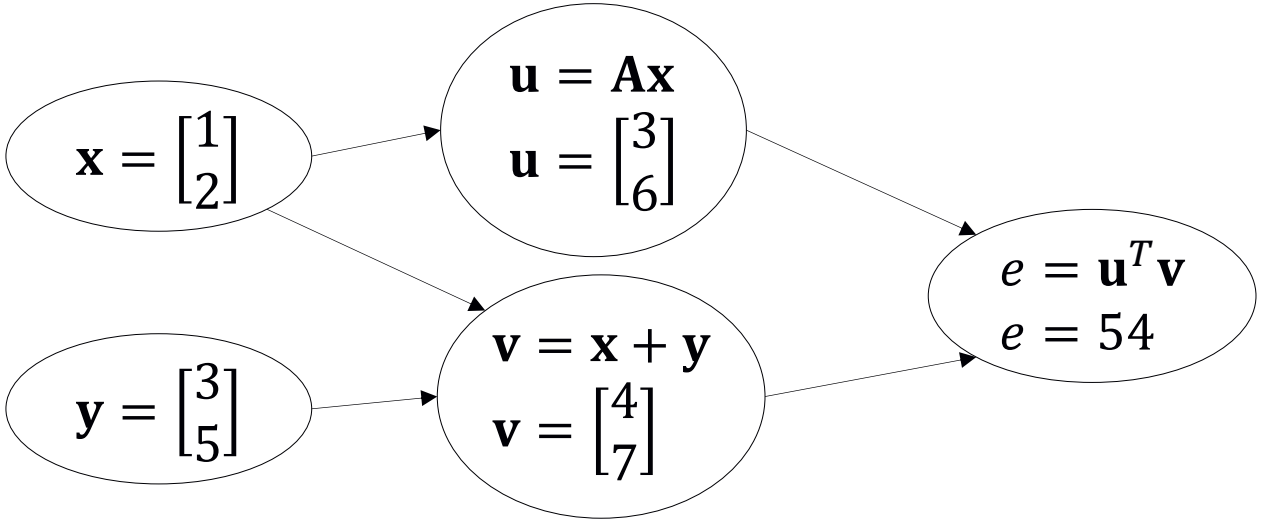
Returning to earlier example



Often, the scalar variables in a computation graph will group naturally into arrays – for example, a 256 x 256 array of light intensities that make up an image.

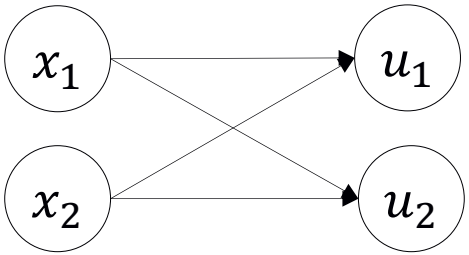
It is convenient and efficient to express the computation graph in terms of these arrays and the elementary functions that operate on them when they are treated as vectors and matrices.

$$e = (\mathbf{A}\mathbf{x})^T(\mathbf{x} + \boldsymbol{y}) \text{ where } \mathbf{A} = \begin{bmatrix} 1 & 1 \\ 2 & 2 \end{bmatrix}$$



Backpropagation on such computation graphs is analogous to the scalar case that we've considered already.

Suppose we have already backpropagated to $\boldsymbol{u}$ and now have the gradient $\begin{bmatrix} \frac{\partial e}{\partial u_1} & \frac{\partial e}{\partial u_2} \end{bmatrix}$. The implicit computation graph underlying the elementary function at $\mathbf{u}$ looks like this:



By the multi-variable chain rule, we propagate the gradient back to the components of x as follows:

Jacobian matrix

$$\begin{bmatrix} \frac{\partial e}{\partial x_1} \\ \frac{\partial e}{\partial x_2} \end{bmatrix} = \begin{bmatrix} \frac{\partial u_1}{\partial x_1}\frac{\partial e}{\partial u_1} + \frac{\partial u_2}{\partial x_1}\frac{\partial e}{\partial u_2} \\ \frac{\partial u_1}{\partial x_2}\frac{\partial e}{\partial u_1} + \frac{\partial u_2}{\partial x_2}\frac{\partial e}{\partial u_2} \end{bmatrix} = \begin{bmatrix} \frac{\partial u_1}{\partial x_1} & \frac{\partial u_1}{\partial x_2} \\ \frac{\partial u_2}{\partial x_1} & \frac{\partial u_2}{\partial x_2} \end{bmatrix}^T \begin{bmatrix} \frac{\partial e}{\partial u_1} \\ \frac{\partial e}{\partial u_2} \end{bmatrix}$$

In the general case, we can express the computation concisely as:

$$\begin{bmatrix} \frac{\partial e}{\partial x_1} \\ \vdots \\ \frac{\partial e}{\partial x_n} \end{bmatrix} = \underbrace{\begin{bmatrix} \frac{\partial u_1}{\partial x_1} & \cdots & \frac{\partial u_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial u_m}{\partial x_1} & \cdots & \frac{\partial u_m}{\partial x_n} \end{bmatrix}^T}_{\textit{Jacobian}\text{ matrix}} \begin{bmatrix} \frac{\partial e}{\partial u_1} \\ \vdots \\ \frac{\partial e}{\partial u_m} \end{bmatrix}$$

This computation lies at the heart of backpropagation in PyTorch.

Finally, remember that we must accumulate gradients as we compute them to allow for different routes to the output.

UNIVERSITY OF LEEDS

Measuring Performance

Deep Learning - COMP5203M

2022-23

Measuring classifier performance

Confusion matrix

		Predicted class		
		Car	Bicycle	Truck
Actual class	Car	8	2	1
	Bicycle	1	7	2
	Truck	2	0	12

Number of trucks classified as cars

$$Accuracy = \frac{\#(correct\ predictions)}{\#(dataset)} = \frac{27}{35}$$

$$Error\ rate = \frac{\#(incorrect\ predictions)}{\#(dataset)} = \frac{8}{35}$$

Special case: binary classification

Two classes: *positive* and *negative (car, no-car)*

		Predicted class	
		Positive	Negative
Actual class	Positive	TP = #(*true positives*)	FN = #(*false negatives*)
	Negative	FP = #(*false positives*)	TN = #(*true negatives*)

$$Accuracy = \frac{TP + TN}{TP + FN + FP + TN}$$

$$Error\ rate = \frac{FP + FN}{TP + FN + FP + TN}$$

Useful to define additional measures that focus on the positive class…

True positive rate and false positive rate

		Predicted class	
		Positive	Negative
Actual class	Positive	TP = #(*true positives*)	FN = #(*false negatives*)
	Negative	FP = #(*false positives*)	TN = #(*true negatives*)

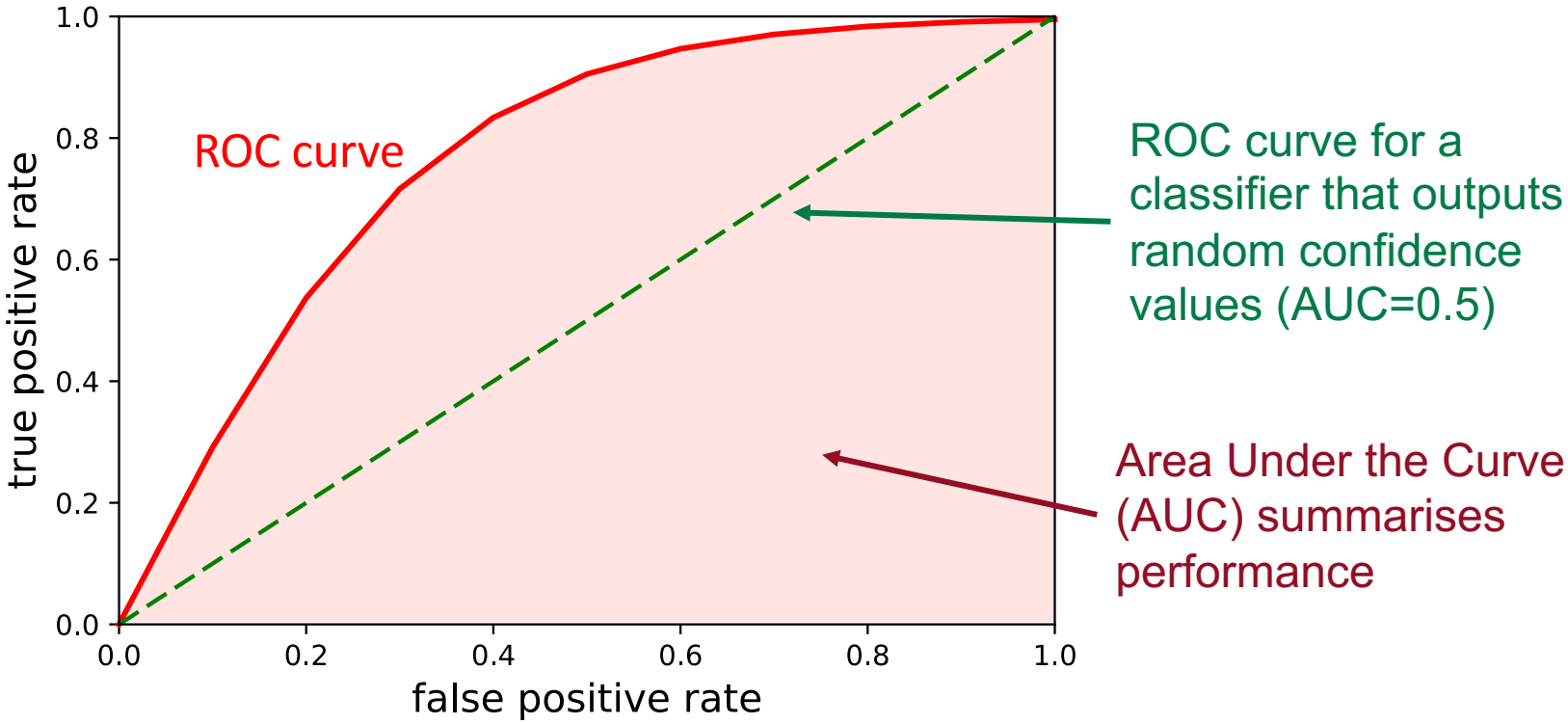
$$true\ positive\ rate\ (TPR) = \frac{TP}{TP + FN}$$ *sensitivity*

$$false\ positive\ rate (FPR) = \frac{FP}{FP + TN}$$ *false alarm rate*

There is a trade-off between the two. By increasing the probability threshold to predict positive, we reduce FPR (good), but also reduce TPR (bad).

Receiver Operator Characteristic (ROC) curve

Visualising the trade-off between TPR and FPR



Precision and Recall

		Predicted class	
		Positive	Negative
Actual class	Positive	10	40
	Negative	90	860

In this case, FPR is uninformative because of the relatively high TN

$$recall = \frac{TP}{TP + FN}$$

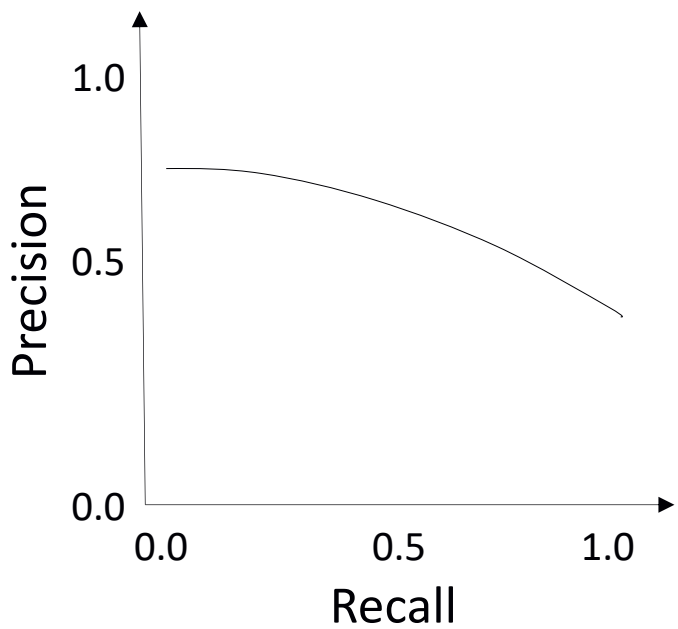
Same as TPR and sensitivity

$$precision = \frac{TP}{TP + FP}$$

Recall is the proportion of <u>actual</u> positives classified correctly
Precision is the proportion of <u>predicted</u> positives classified correctly

Both recall and precision are independent of TN.

Precision-Recall curve



Trade-off precision and recall by varying the threshold probability for predicting positive.

Area Under the Precision-Recall Curve, is a useful measure of classifier performance. The nearer to 1.0 the better.

This is equivalent to *Average Precision*.

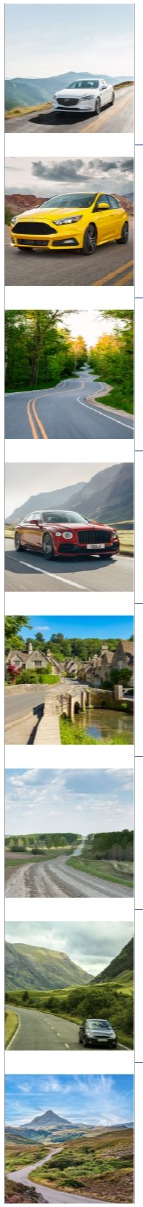
An example

Is there a car in the image?

Classifier outputs a probability of a car being present (positive class)

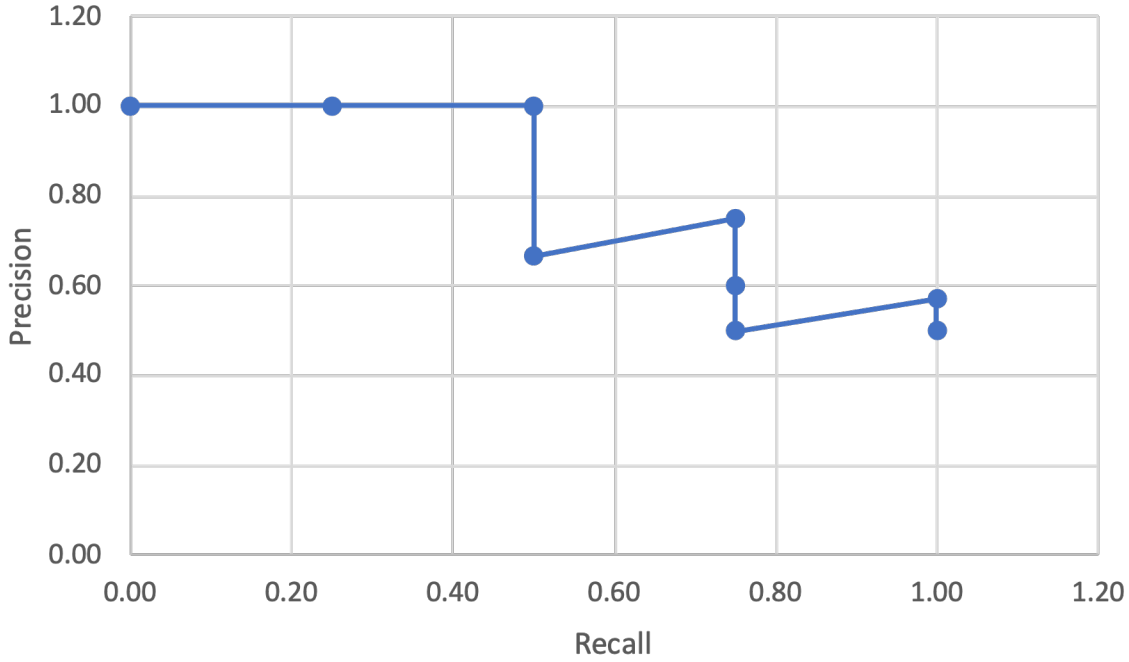
Varying threshold

Increasing probability of positive class



Precision	Recall
1	0
1	1/4
1	1/2
2/ 3	1/2
3/4	3/4
3/ 5	3/4
1/ 2	3/4
4/ 7	1
1/2	1

Precision-recall curve

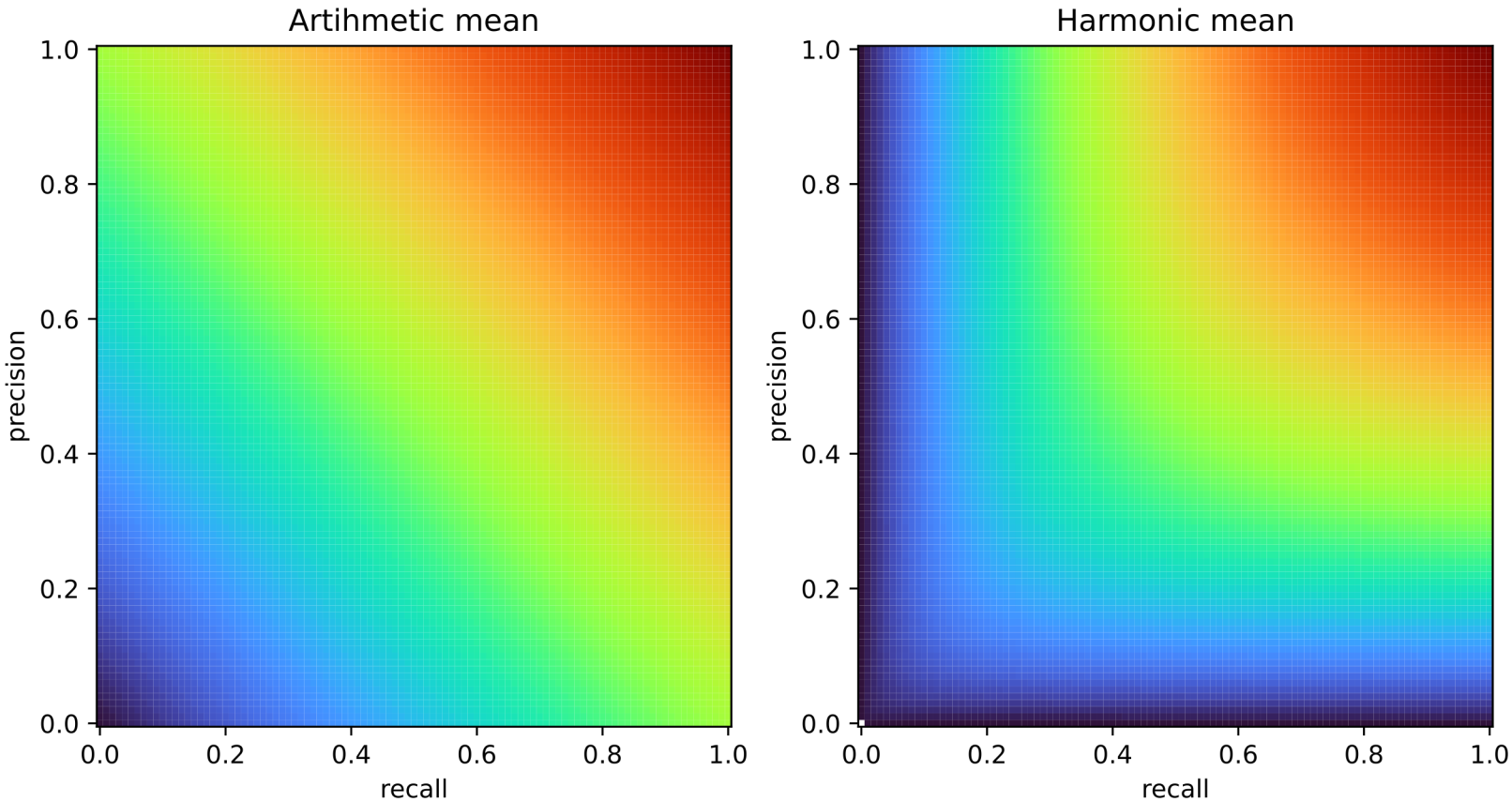


F-score

Summarise precision and recall values as a single value (*harmonic mean*).

$$F = \frac{2}{precision^{-1} + recall^{-1}} = 2 \cdot \frac{precision \cdot recall}{precision + recall}$$

Unlike arithmetic mean, penalises situations in which either precision or recall is small


Artihmetic mean
Harmonic mean
precision
recall
0.0
0.2
0.4
0.6
0.8
1.0

Precision and recall in multiclass classification

$$recall\ for\ truck = \frac{\#(correctly\ predicted\ trucks)}{\#(actual\ trucks)}$$

$$precision\ for\ truck = \frac{\#(correctly\ predicted\ trucks)}{\#(predicted\ trucks)}$$

		Predicted class		
		Car	*Bicycle*	Truck
Actual class	Car	1000	5	5
	Bicycle	10	0	0
	Truck	10	0	0

$$accuracy = \frac{1000}{1030} = 97\%$$

$$recall\ for\ car = \frac{1000}{1010} = 99\%$$

$$recall\ for\ truck = \frac{0}{10} = 0\%$$

$$recall\ for\ bicycle = \frac{0}{10} = 0\%$$

$$balanced\ accuracy = mean\ of\ recall\ values = \frac{0.99 + 0 + 0}{3} = 0.33$$